To appear in: IEEE Robotics & Automation Magazine

The Open Motion Planning Library 2.0

Weihang Guo[1], Theodoros Tyrovouzis[1], Emiliano Flores[1], Clayton W. Ramsey[1], Zachary K. Kingston[2*], Ioan A. Şucan[3*], Mark Moll[4], Lydia E. Kavraki[1,5]

***Abstract*—The Open Motion Planning Library (OMPL), first released in 2008, has become a cornerstone of the motion planning community, providing implementations of a wide range of state-of-the-art sampling-based algorithms. Over almost two decades of continuous development, we have steadily expanded the library with new planners, state spaces, and problem formulations. These additions range from asymptotically optimal and lazy planners to constrained motion planning and planning with temporal-logic goals. Building on this foundation, we introduce OMPL 2.0, a major evolution of the library that targets real-time motion planning through hardware acceleration and integrates seamlessly with modern AI research workflows. We also reflect on how OMPL and the field of motion planning have grown together over the years, and discuss the library's broader impact on the research community.**

I. Introduction

The Open Motion Planning Library (OMPL) is an open-source framework for sampling-based motion planning [1]. First released in 2008, OMPL provides a broad collection of sampling-based planning algorithms [2] together with reusable abstractions for representing state spaces, specifying planning problems, and evaluating solutions. By separating motion planning from robot models, collision checking, and simulation, OMPL can be integrated with a wide range of robotic systems while allowing researchers to develop and compare planning algorithms within a common framework.

For nearly two decades, OMPL has evolved continuously alongside advances in motion planning research and the changing needs of the robotics community. Across more than 30 releases, the library has incorporated new planners, state-space representations, problem formulations, benchmarking tools, and software integrations. Sustaining OMPL as research infrastructure has also required continued investment in software maintenance, documentation, reproducible evaluation, and community support.

OMPL 2.0 marks a focused modernization within this longer trajectory. It introduces hardware-efficient planning and collision checking through VAMP [3] and CAPT [4], transitions the Python bindings to nanobind [5, 6], modernizes the library's development and distribution infrastructure, and streamlines its integration with the broader robotics software ecosystem. In this article, we first describe the core design that has enabled OMPL to remain extensible, then review its evolution over nearly two decades, highlight the principal advances in OMPL 2.0, and discuss the community and research directions that will shape its future.

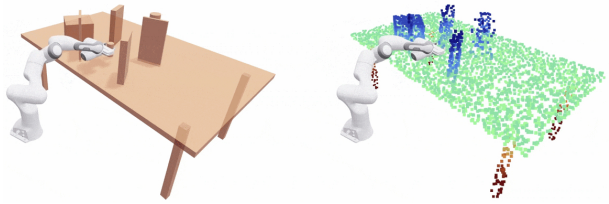


Fig. 1: OMPL 2.0 supports planning with realistic robots and environments represented by meshes or point clouds.

II. The Open Motion Planning Library

Before introducing new features of OMPL 2.0, we recap the main components of OMPL. This design, introduced in the original OMPL paper [1], remains largely unchanged and has stood the test of time. OMPL is structured as a set of modular components that correspond closely to the fundamental concepts of sampling-based motion planning. The core abstractions include state space, state validator, state sampler, start and goal, and motion planners, which together define the planning problem.

The *state space* defines the representation and topology of the robot's configuration space, along with its limits and any kinematic or dynamic constraints. A *state validator* provides a generic interface for collision checking, allowing users to connect any collision checking library or simulator. Finally, a *state sampler* draws configurations uniformly or from a Gaussian distribution over the state space. With these foundational components in place, users can select or build their own *motion planner* and specify *start* and *goal* states to solve the problem. Beyond solving individual planning problems, the *benchmark* component provides extensible infrastructure for running evaluations and analysis across multiple planners [7]. Results and metadata are stored in a database to support reproducibility.

A key design choice in OMPL is to focus exclusively on motion planning algorithms and to not include specific representations of robots, workspaces, or collision detection. Instead, OMPL relies on external software components to provide these capabilities. This minimalist design allows the library to remain general and easily integrated with a wide range of robotics systems and simulation environments. OMPL is a community effort, with contributions from many researchers over the years.

[1]WG, TT, EF, CWR, LEK are with the Department of Computer Science, Rice University, Houston TX, USA. {wg25,tt88,ef55,cwr3,kavraki}@rice.edu

[2]ZKK is with the Department of Computer Science, Purdue University, West Lafayette, IN. zkingston@purdue.edu

[3]IAŞ is with Waymo, LLC, Mountain View, CA

[4]MM is with Metron, Inc., Reston, VA

[5]LEK is also affiliated with the Ken Kennedy Institute at Rice University

*Work by these authors was done while they were at Rice University.

III. Two Decades of Evolution and Adoption

OMPL's core abstractions have remained stable, but the library built around them has evolved continuously. Since the publication of the original paper, more than 30 releases have expanded OMPL with new algorithms and capabilities in response to advances in motion planning research and the changing needs of the robotics community. Many capabilities now associated with OMPL are therefore the result of sustained development rather than a single release.

Over time, the planner collection has grown to include asymptotically optimal planners [8–10] and lazy planning methods [11]. The range of planning problems that OMPL can express has expanded as well. Constrained motion planning [12] enables sampling-based planners to operate on implicit constraint manifolds, supporting problems involving contact constraints and closed kinematic chains. LTLPlanner [13] supports kinodynamic planning with linear temporal logic goals. The library has also added discrete and time-augmented state spaces, as well as non-Euclidean representations such as Dubins and Reeds-Shepp spaces for car-like vehicles. As the number of planners and parameters has grown, hyperparameter tuning has provided an effective tool for planner selection and configuration [14].

OMPL has become a widely used tool in the motion planning community. The original paper has accumulated over 2,500 citations, reflecting sustained and widespread adoption. By providing carefully maintained reference implementations and standardized benchmarking infrastructure [7], OMPL allows researchers to evaluate new algorithms without repeatedly reimplementing baselines.

OMPL's modular architecture has been central to its broad adoption, enabling integration with systems across the robotics ecosystem. OMPL integrates with MoveIt [15], robot simulators such as CoppeliaSim [16] and Flightmare [17], industrial frameworks such as Tesseract [18], and research toolkits including AIKIDO [19], EXOTica [20], and the Kautham Project [21]. OMPL's planners have also been applied to task-and-motion planning [22] and mobile robot navigation through Nav2 [23]. Tools such as HyperPlan [14], Robowflex [24], and MotionBenchMaker [25] build on its benchmarking infrastructure, while applications beyond robotics include modeling the conformational flexibility of large macromolecules [26].

IV. OMPL 2.0: A Focused Modernization

Building on nearly two decades of continuous development, OMPL 2.0 brings together a focused set of modernization efforts. It preserves OMPL's core planning abstractions while introducing hardware-efficient planning, a modern Python interface and distribution workflow, and a streamlined role within the broader robotics software ecosystem.

a) Hardware-Efficient Planning: OMPL 2.0 integrates VAMP [3], which leverages CPU SIMD instructions to perform collision checking and forward kinematics with fine-grained, hardware-efficient parallelism. With this integration, planners can find solutions in microseconds, and aggregate solution rates can reach the kilohertz range, all without specialized hardware such as a GPU. OMPL 2.0 also includes CAPT [4], which extends the SIMD approach to point cloud collision checking. These capabilities support applications that require extremely fast planning in dynamic environments, including reactive planning and task-and-motion planning. OMPL 2.0's integration with VAMP and CAPT also makes it straightforward to apply motion planning to realistic robots operating in real-world scenes (see Fig. 1).

b) Modern Python Interface and Distribution: Earlier versions of OMPL used Py++ [27] to generate Python bindings, but that tool is no longer maintained. OMPL 2.0 has transitioned to nanobind [5], a modern C++ binding framework that provides a lightweight and efficient interface between C++ and Python while simplifying the maintenance of binding code [6]. OMPL 2.0 also uses GitHub Actions for automated building and testing across multiple platforms, together with automated pipelines that build and publish Python wheels to PyPI. These changes make the library easier to install, distribute, and integrate into modern Python-based robotics and AI workflows.

c) Modernized Integration: Earlier versions of OMPL included OMPL.app, a graphical front end that demonstrated integration with external libraries for mesh loading, collision checking, and visualization. The robotics ecosystem now provides widely used simulation platforms and collision detection libraries that integrate naturally with OMPL, making a dedicated graphical interface unnecessary. OMPL.app has therefore been removed in OMPL 2.0, with demos migrating to Viser [28], Pinocchio [29], and other external robotics software.

V. Project Community and Organization

OMPL is sustained as open-source infrastructure for research, education, and real-world robotics. Maintaining its relevance requires long-term software support, documentation, and community engagement, all of which remain challenging in an academic environment. Tutorials have introduced researchers to OMPL since its early releases, while improved documentation is now broadly accessible for undergraduates, motion planning researchers, practitioners, and everyone in between.[1]

We also decided early on to spend significant time on building an extensible infrastructure for benchmarking motion planning algorithms [7]. This has been invaluable in quantifying how new planners advance the state of the art. More recently, we have also generated large datasets of realistic hard motion planning problems (and made the tooling for generating such datasets available as well) [25]. Benchmark results can be visualized through an interactive browser-based tool called Planner Arena.[2] Fig. 2 shows sample results for a parameterized benchmark. The parameters in this case are (1) the specific motion planning problem, (2) how the environment is represented (mesh vs. point cloud), and (3) the type of robot (a Fetch, a Panda, and a UR5 in this example). The UI enables comparison of planning algorithms on all robots for a single problem, on a single robot on all problems, etc. In each case,

[1] https://kavrakilab.org/icra-2026-ompl-tutorial/
[2] https://planner-arena.kavrakilab.rice.edu/

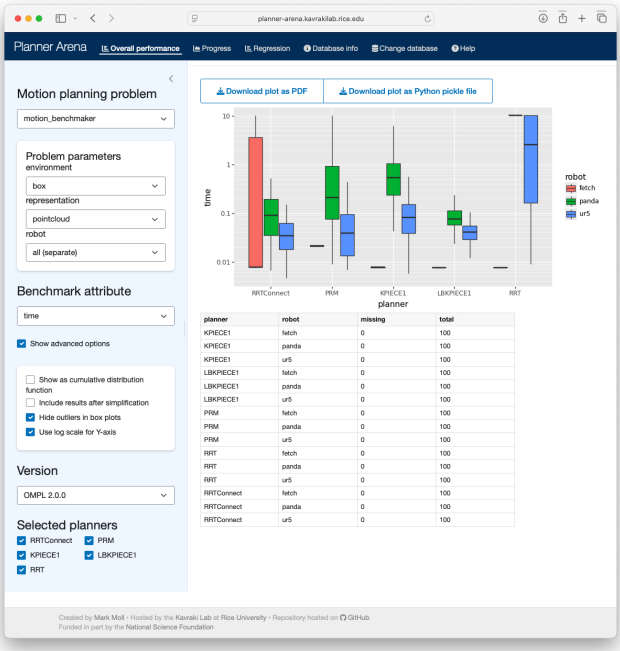


Fig. 2: Planner Arena allows users for benchmarking motion planning algorithms.

there are many performance metrics that can be visualized, such as planning time and solution path length. The table in the bottom right shows the number of missing values: if a planner was not able to find a path on some runs, the path length would be considered missing for those runs. In summary, Planner Arena encourages exploration and a nuanced characterization of planner performance.

The abstractions that OMPL imposes on planning algorithms are minimal and make it straightforward to develop prototypes of planners. Evaluation in simplified environments (e.g., a point robot in 2D) makes testing easy, before deployment in realistic environments without having to change the planning algorithm implementation itself.

Many motion planning researchers have developed new motion planning algorithms using OMPL. Oftentimes, the original authors contributed their implementations to the OMPL repository. For example, Karaman and Frazzoli contributed the initial RRT* implementation to OMPL shortly after publication [8]. Bekris et al. contributed implementations of their asymptotically near-optimal algorithms [30]. Gammell et al. have contributed many algorithms such as BIT*, ABIT*, AIT*, EIT*, and AORRTC [9, 31]. Orthey et al. have contributed implementations of their work on multi-level planning [32] and planning in space-time [33]. This is by no means an exclusive list.[3]

Over time, the OMPL community has grown to include many AI researchers, who often use OMPL simply as a black box. Because Python is the primary language for integrating software components in the AI community, OMPL's Python bindings have become increasingly important. Originally, these bindings were intended mainly to lower the barrier to entry for beginning programmers. Today, their performance penalty is small enough to be acceptable for many use cases, though we still recommend the C++ API directly when optimal performance is needed.

[3]A list of contributors is maintained in https://ompl.kavrakilab.org/developers.html

VI. The Next Decade of OMPL

Looking ahead, we see several directions that will shape the future development of OMPL. Motion planning is increasingly used as a component within larger decision-making systems: task and motion planning [22] combines symbolic reasoning with geometric planning, planning under uncertainty, such as POMDPs [34], demands rapid replanning as beliefs evolve, and reactive planning [35] requires algorithms that quickly adapt to changing environments. Supporting these paradigms will require continued investment in fast, anytime planning and interfaces that facilitate integration with higher-level reasoning. Beyond robotics pipelines, OMPL can also serve the broader AI ecosystem. By providing a Model Context Protocol server and agent skills, we aim to let LLM-based agents invoke motion planning directly as a tool.

As robotic systems scale from single manipulators to teams of cooperating robots, multi-robot planning in composite configuration spaces becomes a central challenge. Recent work has shown that sampling-based methods can be extended to these settings by exploiting problem structure [36], and future versions of OMPL aim to provide support for multi-robot coordination. At the same time, GPU-parallel approaches to tree construction and collision checking [37] offer the potential for massive speedups. Finally, emerging applications in soft and growing robots [38] involve high-dimensional, continuously deformable state spaces that challenge existing representations and will require extending OMPL's state space abstractions to new physical domains.

These directions share a common theme: the core abstractions of sampling-based planning remain the right foundation, but must be extended to meet the demands of increasingly complex robotic systems. We see OMPL 2.0 not as an endpoint, but as a starting point for the next decade of motion planning research.

Acknowledgements

The authors gratefully acknowledge the many contributors whose work over the years has shaped the OMPL 2.0. All contributors are acknowledged on the OMPL web page. Contributions can be found in the OMPL GitHub repository. We note, however, that commit counts and lines of code are imperfect indicators of impact. Several significant contributions, such as the initial RRT* implementation, predate the move to GitHub and do not appear in those statistics. The authors would like to thank current and former members of the Kavraki Lab, as well as the broader motion planning community, whose discussions, feedback, and support have been instrumental to this work. This work was supported in part by NSF OAC-2411219. Theodoros Tyrovouzis and Lydia E. Kavraki have been supported in part by ERDC W912HZ2320003.

REFERENCES

[1] I. A. Şucan, M. Moll, and L. E. Kavraki. "The Open Motion Planning Library". In: *IEEE Robotics and Automation Magazine* 19.4 (Dec. 2012). https://ompl.kavrakilab.org, pp. 72–82.

[2] A. Orthey, C. Chamzas, and L. E. Kavraki. "Sampling-based motion planning: A comparative review". In: *Annual Review of Control, Robotics, and Autonomous Systems* 7 (2023).

[3] W. Thomason, Z. Kingston, and L. E. Kavraki. "Motions in Microseconds via Vectorized Sampling-Based Planning". In: *IEEE International Conference on Robotics and Automation*. 2024, pp. 8749–8756.

[4] C. W. Ramsey, Z. Kingston, W. Thomason, and L. E. Kavraki. "Collision-Affording Point Trees: SIMD-Amenable Nearest Neighbors for Fast Collision Checking". In: *Robotics: Science and Systems*. 2024.

[5] W. Jakob. *nanobind: tiny and efficient C++/Python bindings*. https://github.com/wjakob/nanobind. 2022.

[6] W. Guo, T. Tyrovouzis, and L. E. Kavraki. "Python Bindings for a Large C++ Robotics Library: The Case of OMPL". In: *IEEE International Conference on Robotics and Automation*. 2026.

[7] M. Moll, I. A. Şucan, and L. E. Kavraki. "Benchmarking Motion Planning Algorithms: An Extensible Infrastructure for Analysis and Visualization". In: *IEEE Robotics & Automation Magazine (Special Issue on Replicable and Measurable Robotics Research)* 22.3 (Sept. 2015), pp. 96–102.

[8] S. Karaman and E. Frazzoli. "Sampling-based algorithms for optimal motion planning". In: *The International Journal of Robotics Research* 30.7 (2011), pp. 846–894.

[9] T. S. Wilson, W. Thomason, Z. Kingston, and J. D. Gammell. "AORRTC: Almost-Surely Asymptotically Optimal Planning With RRT-Connect". In: *IEEE Robotics and Automation Letters* 10.12 (2025), pp. 13375–13382.

[10] J. D. Gammell, T. D. Barfoot, and S. S. Srinivasa. "Batch informed trees (BIT*): Informed asymptotically optimal anytime search". In: *The International Journal of Robotics Research* 39.5 (2020), pp. 543–567.

[11] R. Bohlin and L. Kavraki. "Path planning using lazy PRM". In: *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation*. Vol. 1. 2000, 521–528 vol.1.

[12] Z. Kingston, M. Moll, and L. E. Kavraki. "Exploring Implicit Spaces for Constrained Sampling-Based Planning". In: *International Journal of Robotics Research* 38.10–11 (Sept. 2019), pp. 1151–1178.

[13] A. Bhatia, M. R. Maly, L. E. Kavraki, and M. Y. Vardi. "Motion planning with complex goals". In: *IEEE Robotics & Automation Magazine* 18.3 (2011), pp. 55–64.

[14] M. Moll, C. Chamzas, Z. Kingston, and L. E. Kavraki. "HyperPlan: A framework for motion planning algorithm selection and parameter optimization". In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE. 2021, pp. 2511–2518.

[15] S. Chitta, I. Şucan, and S. Cousins. "MoveIt! [ROS Topics]". In: *IEEE Robotics and Automation Magazine* 19.1 (2012), pp. 18–19.

[16] E. Rohmer, S. P. Singh, and M. Freese. "V-REP: A versatile and scalable robot simulation framework". In: *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE. 2013, pp. 1321–1326.

[17] Y. Song, S. Naji, E. Kaufmann, A. Loquercio, and D. Scaramuzza. "Flightmare: A Flexible Quadrotor Simulator". In: *Conference on Robot Learning*. 2020.

[18] L. Armstrong. *Tesseract: Motion Planning Environment*. https://github.com/tesseract-robotics/tesseract. 2018.

[19] M. Koval, J. King, J. Lee, G. Lee, B. Hou, P. Velagapudi, C. Liddick, D. Yi, A. V. Mandalika, S. Niyaz, E. Gordon, et al. *AIKIDO: Artificial Intelligence for Kinematics, Dynamics, and Optimization*. Version 0.4.0. 2020.

[20] V. Ivan, Y. Yang, W. Merkt, M. P. Camilleri, and S. Vijayakumar. "EXOTica: An Extensible Optimization Toolset for Prototyping and Benchmarking Motion Planning and Control". In: *Robot Operating System (ROS): The Complete Reference (Volume 3)*. Ed. by A. Koubaa. Cham: Springer International Publishing, 2019, pp. 211–240.

[21] J. Rosell, A. Pérez, A. Aliakbar, Muhayyuddin, L. Palomo, and N. García. "The Kautham Project: A Teaching and Research Tool for Robot Motion Planning". In: *IEEE International Conference on Emerging Technologies and Factory Automation*. 2014, pp. 1–8.

[22] N. T. Dantam, Z. K. Kingston, S. Chaudhuri, and L. E. Kavraki. "An incremental constraint-based framework for task and motion planning". In: *The International Journal of Robotics Research* 37.10 (2018), pp. 1134–1151.

[23] S. Macenski, F. Martín, R. White, and J. G. Clavero. "The marathon 2: A navigation system". In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE. 2020, pp. 2718–2725.

[24] Z. Kingston and L. E. Kavraki. "Robowflex: Robot motion planning with MoveIt made easy". In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE. 2022, pp. 3108–3114.

[25] C. Chamzas, C. Quintero-Peña, Z. Kingston, A. Orthey, D. Rakita, M. Gleicher, M. Toussaint, and L. E. Kavraki. "MotionBenchMaker: A Tool to Generate and Benchmark Motion Planning Datasets". In: *IEEE Robotics and Automation Letters* 7.2 (Apr. 2022), pp. 882–889.

[26] B. Gipson, M. Moll, and L. E. Kavraki. "SIMS: A hybrid method for rapid conformational analysis". In: *PLOS ONE* 8.7 (2013). PMCID: PMC3720858, PMID: 23935893, p. 68826.

[27] R. Yakovenko. *Py++ - Boost.Python code generator*. https://github.com/ompl/pyplusplus. 2004.

[28] B. Yi, C. M. Kim, J. Kerr, G. Wu, R. Feng, A. Zhang, J. Kulhanek, H. Choi, Y. Ma, M. Tancik, and A. Kanazawa. "Viser: Imperative, Web-based 3D Visualization for Python". In: *Robotics: Science and Systems*. 2026.

[29] J. Carpentier, G. Saurel, G. Buondonno, J. Mirabel, F. Lamiraux, O. Stasse, and N. Mansard. "The Pinocchio C++ library – A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives". In: *IEEE International Symposium on System Integrations*. 2019.

[30] Y. Li, Z. Littlefield, and K. E. Bekris. "Asymptotically Optimal Sampling-Based Kinodynamic Planning". In: *International Journal of Robotics Research* 35 (Apr. 2016), pp. 528–564.

[31] M. P. Strub and J. D. Gammell. "Adaptively Informed Trees (AIT*) and Effort Informed Trees (EIT*): Asymmetric bidirectional sampling-based path planning". In: *The International Journal of Robotics Research* 41.4 (2022), pp. 390–417.

[32] A. Orthey, S. Akbar, and M. Toussaint. "Multilevel motion planning: A fiber bundle formulation". In: *The International Journal of Robotics Research* 43.1 (2024), pp. 3–33.

[33] F. Grothe, V. N. Hartmann, A. Orthey, and M. Toussaint. "ST-RRT*: Asymptotically-Optimal Bidirectional Motion Planning through Space-Time". In: *IEEE International Conference on Robotics and Automation*. 2022, pp. 3314–3320.

[34] Y. Liang, E. Kim, W. Thomason, Z. Kingston, L. E. Kavraki, and H. Kurniawati. "Think Fast and Far: Long-Horizon Online POMDP Planning via Rapid State Sampling". In: *International Journal of Robotics Research* (2026).

[35] M. Bhardwaj, B. Sundaralingam, A. Mousavian, N. D. Ratliff, D. Fox, F. Ramos, and B. Boots. "STORM: An integrated framework for fast joint-space model-predictive control for reactive manipulation". In: *Conference on Robot Learning*. PMLR. 2022, pp. 750–759.

[36] W. Guo, Z. Kingston, K. Hang, and L. E. Kavraki. "Efficient Multi-Robot Motion Planning for Manifold-Constrained Manipulators by Randomized Scheduling and Informed Path Generation". In: *IEEE Robotics and Automation Letters* 11.4 (2026), pp. 4385–4392.

[37] C. H. Huang, P. Jadhav, B. Plancher, and Z. Kingston. "pRRTC: GPU-parallel RRT-Connect for fast, consistent, and low-cost motion planning". In: *IEEE International Conference on Robotics and Automation*. 2026.

[38] Y. Gao, L. Chen, P. Bhovad, S. Wang, Z. Kingston, and L. H. Blumenschein. "Parallel Simulation of Contact and Actuation for Soft Growing Robots". In: *Soft Robotics* (2026).